

Project ESCAPE

Entropy & Semantic Compute Allocation in Programming Environments: Structural Discovery
via Entropy in the Byte Latent Transformer Across Code and Natural Language

Priyanka Agarwal (2024101016)

Shashwat Mukadam (2024102008)

Divyansh Atri (2024113001)

Team Trimax · CS7.501 Advanced NLP · Final Project Proposal

Abstract

Language models split text with a vocabulary fixed before training, so boilerplate and dense logic are chopped alike. The Byte Latent Transformer (BLT) [Pagnoni et al., 2024] instead reads raw bytes and splits *as it goes*, cutting wherever its own next-byte prediction becomes uncertain: each cut is the model reporting where its knowledge ran out. Nobody has checked whether those cuts mean anything. We ask a narrow, falsifiable version — **do BLT’s boundaries land at the start of syntactic constituents in code?**, the Grammatical Branching Hypothesis. We give a mechanism for why they should, state where it must fail, and test it against a null model holding boundary density fixed, so a positive result is not an artifact of BLT’s threshold τ . Prose is a built-in negative control: if code and prose do not separate, we are measuring τ , not grammar. We further test whether an entropy-only mechanism can be mistaken for semantic risk-awareness in memory-unsafe C++, and show the two can be told apart. A null result is as informative as a positive one.

1 Introduction

Tokenization has been the standard front end for language models for a decade, but it fixes a compression ratio in advance: a tokenizer trained before the model ever sees your data spends as many tokens on a line of boilerplate as on a line of subtle logic. BLT removes the fixed vocabulary. A small byte-level language model — the *patcher* — reads raw bytes and measures, at each position, its uncertainty about the next byte, i.e. Shannon entropy:

$$H(b_i) = - \sum_v p(v \mid b_{<i}) \log p(v \mid b_{<i}) \quad (1)$$

When H crosses a threshold τ , BLT starts a new *patch*; a large transformer then processes those variable-length patches, spending more compute where bytes were hard to predict and less where they were easy.

What matters here is that a boundary is not a pre-processing decision. **It is a claim the model makes about its own uncertainty** — interpretable in a way a BPE merge table never is, because we can ask whether the claim corresponds to anything independently verifiable. The BLT authors scale the approach to 8B parameters on 4T training bytes but report only efficiency and downstream accuracy; they never check

whether the induced patches line up with structure we can check. That is the gap. Code is the right place to look: it carries machine-checkable ground truth in its abstract syntax tree (AST), and a grammar with sharper choice points than flexible prose.

Contributions. (1) **A stated mechanism, not a black-box metric:** a falsifiable account (§4.2) of *why* boundaries should fall at constituent starts, including where it must fail. (2) **A null model that makes the result mean something:** a density-matched baseline plus a whitespace baseline (§4.3), so alignment is attributable to grammar, not to boundary density or newlines. (3) **A built-in negative control:** the same mechanism predicts a far weaker effect in prose, so failure to separate code from prose is itself evidence we are measuring τ . (4) **An honest scope limit:** we test what an entropy-only mechanism *cannot* see (§4.5) rather than overclaiming that surface predictability implies danger-awareness.

2 Motivation and Related Work

Interpretability work on tokenization has focused almost entirely on *static* vocabularies. Byte-pair encoding [Sennrich et al., 2016] fixes a merge table before training, and its failure modes — poor morphological alignment, unstable segmentation of rare or multilingual tokens — are well documented. Tokenization-free models remove the vocabulary but not the uniformity: CANINE [Clark et al., 2022] and ByT5 [Xue et al., 2022] spend equal compute on every byte, so they never decide *where* a boundary belongs. BLT is the first architecture at scale where segmentation is both byte-level and dynamic — exactly what makes its boundaries worth probing.

Structural probing on code, such as CodeBERT [Feng et al., 2020], and syntax probes on contextual representations [Hewitt and Manning, 2019] show that *learned representations* can encode syntax — but in every case the model was trained or probed *for* that structure. None asks whether an unsupervised, compression-only signal, with no representation-learning objective and no exposure to a parser, recovers syntax as a side effect. Two gaps follow: whether BLT’s patcher implicitly recovers constituent structure, and whether such recovery is genuine grammar-tracking or merely an artifact of patch density being pinned by τ — a distinction requiring a principled null model, not a raw overlap score.

3 Research Objectives

- O1. Syntactic Alignment.** Do BLT’s entropy-triggered boundaries align with AST node boundaries in C++ and Python, beyond a density-matched baseline?
- O2. Compute Allocation Ratio.** How do mean Bytes-Per-Patch (BPP) and its variance differ between rigid code syntax and natural language?
- O3. Cross-Language Structural Recognition.** Does patching differ for indentation-based (Python) versus bracket-based (C++) syntax?
- O4. Robustness to Code Noise.** Do boundaries survive character-level noise (syntax errors, obfuscation), or does the mechanism fail systemically?
- O5. Semantic vs. Surface Compute Allocation.** Is extra compute given to memory-unsafe C++ operations *fully explained* by the surface unpredictability of nearby identifiers and casts, or does a residual remain after conditioning on that confound?

4 Hypothesis

4.1 The Grammatical Branching Hypothesis (H1)

BLT’s patch boundaries are not random compression artifacts: they concentrate at the **start** of syntactic constituents rather than inside them, more strongly in code than in prose, and more strongly at starts than at ends. Precisely, H1 claims an *average-case asymmetry* between constituent starts and constituent interiors. It does **not** claim that starts are points of maximal grammatical choice; §4.2 explains why that stronger claim is false and why it is not needed. We measure how large the asymmetry is.

4.2 Why We Expect This to Hold

The patcher is a small byte-level language model with a short (512-byte) context window that emits a boundary whenever next-byte entropy crosses τ . A boundary therefore marks exactly one thing: **a position where the preceding context stops determining what comes next**. Everything below follows from that sentence.

Entropy bounds branching; it does not track it.

It is tempting to read entropy as the local branching factor B , the number of grammatically valid continuations. It is not: entropy depends on the *shape* of the distribution, not the size of its support. The only guaranteed relation is $H \leq \log B$, and both failure directions are real: ten continuations with one holding 99% of the mass give $H = 0.08$ nats, far below our $\tau = 1.34$, while four equiprobable ones give $H = 1.39$ nats and do cross it. Branching is necessary for high entropy but nowhere near sufficient, and a flat distribution over few options outranks a peaked one over many.

The bound survives in the useful direction. A boundary *certifies* that the local distribution was genuinely flat: $H > \tau$ implies $B > e^\tau \approx 3.8$, so every entropy-triggered boundary sits where at least four continuations were live. Boundaries are therefore a **subset** of high-branching positions — the subset where the grammar has not already concentrated its

Position	Entropy	Split?	Case
i of <code>if</code>	high	yes	correct
(after <code>if</code>	low	no	<i>missed</i> start
x of <code>x > 0</code>	high	yes	lexical, not syntactic
) ; }	low	no	correct (supports P2)

Table 1: The mechanism applied to `if (x > 0) { total += 1; }`. Row 1 is the effect we predict; row 2 costs recall (forced opener); row 3 costs precision (identifier entropy); row 4 is why starts should beat ends.

mass on one continuation. Hence a prediction we state in advance: **precision against constituent starts should exceed recall**.

The load-bearing claim is about interiors, not starts. Our argument nowhere requires constituent starts to be maximal-choice points, only that entropy be lower *inside* a committed production than at its boundary — a structural property of context-free grammars, not a statistical hope. Once a production is entered its own rule constrains what may follow: mid-keyword and mid-identifier bytes are cheap, and closing delimiters (`;`, `)`, `}`) are among the most predictable bytes in a file because the grammar *requires* them once the production is committed — they are not chosen. This is the asymmetry P2 rests on: boundaries are pushed *away from* interiors and right edges, and constituent starts inherit them by elimination rather than by being choice-maximal.

Where this mechanism does not hold. Two dissociations follow; they bound what a positive result can mean, and they damage *different* metrics. First, some constituents open deterministically: in both C++ and Python, `if`, `for` and `while` take a forced next token, and `def` must be followed by a name. These are constituent starts with branching factor near one, and the mechanism predicts BLT will miss them — a **recall** cost, tested directly by P4. Second, high entropy arises where no constituent opens at all: part-way through an identifier, inside a string literal, inside a comment. These are lexical choice points, not grammatical ones — a **precision** cost, and the confound isolated in §4.5. The patcher measures next-byte predictability, conflating grammatical branching with lexical arbitrariness, and nothing in the architecture separates them. Table 1 walks one line of C++ through all three cases.

Prose is the contrast case. Branching is flatter across a sentence: content words are hard everywhere, function words easy everywhere, and no formal rule forces a clause to end with a specific character. Prose should therefore show more uniform patch lengths and weaker structural anchoring. **This code/prose gap is what makes the claim one about grammar rather than about τ .**

4.3 Predictions and Null Model

P1: Alignment. Boundary positions match AST node start offsets, within a tolerance k fixed *a priori* per language on a held-out calibration split, well above baseline, for statement- and expression-level nodes.

P2: Asymmetry. Alignment at node starts is clearly higher than at node ends: closing delimiters are low-entropy, so BLT should systematically miss right edges. Span-IoU therefore *understates* the effect, so boundary precision/recall against start offsets — not IoU — is our primary metric.

P3: Cross-domain contrast. BPP variance is high in code and low in prose, and P1-style alignment in prose (against a constituency parse) is near chance. Code and prose must dissociate.

P4: Stratification by opener. Alignment is measurably weaker for constituent types with deterministic openers (`if.statement`, `for.statement`) than for open-ended ones (`expression.statement`, `return.statement`, assignment right-hand sides). P4 follows from §4.2 and is the sharpest test that the mechanism running is ours: a generic “entropy correlates with structure” story does not predict it.

Null model (H0). Agreement between patch boundaries and AST node starts is no better than a density-matched random boundary set: the same number of boundaries, patch lengths drawn from BLT’s own empirical patch-length distribution, in shuffled order. This baseline is required, not optional — mean patch length is roughly pinned by τ , so *any* boundary set of the right density scores nonzero raw alignment, and only the margin over H0 is informative. We assess it by permutation test (10,000 resamples), reporting effect size with confidence intervals rather than a raw overlap number.

Second baseline: whitespace. H0 is structure-blind by construction, so it cannot rule out the most obvious confound in code: most AST node starts sit immediately after a newline and indentation, so a trivial “split at every newline” rule would score well on P1. It is therefore an explicit second baseline: H1 is only interesting if BLT beats *that*, not merely chance. This matters most in Python, where indentation *is* the grammar.

Depth control. Alignment is additionally computed *per AST depth*: every byte sits inside several nested nodes, so “alignment” is undefined until granularity is fixed.

What counts as a result. H1 is supported if P1 clears both baselines with non-overlapping confidence intervals, P2 holds in the predicted direction, P3 shows code/prose dissociation, and P4’s stratification appears. Any subset failing is reported as such, and each failure mode is itself interpretable.

4.4 Why the Result Is Not Obvious in Advance

If H1 holds, an unsupervised, entropy-driven compressor recovers syntactic constituent boundaries with no parser in the loop — an interpretability result about what dynamic patching has implicitly learned, and evidence that BLT transfers to code without a code-specific tokenizer.

If H1 fails, *how* it fails is the signal. Chance alignment with uniformly **high** entropy means the patcher is undertrained on code and allocates compute blind

to grammar — a reportable limitation with a clear fix: retrain the entropy model on code. Chance alignment with uniformly **low** entropy means code is so compressible that τ rarely fires, making boundaries density-limited rather than structure-driven. H1 could also lose outright if identifier entropy dominates: names are arbitrary strings, not grammar-constrained, and could swamp the syntactic signal — the precision failure above, addressed in §4.5. Specifying distinct, diagnosable failure modes rather than a single pass/fail outcome is what makes the experiment informative either way.

4.5 Scope Limit: What Entropy Cannot See

The patcher is a short-context, surface-statistics model with no representation of pointer aliasing, ownership, or memory safety. Tokens like `new`, `delete`, `*` and `&` are short, frequent and lexically predictable, so §4.2 predicts **low** entropy at them, not high. Consequently **we do not expect BLT to allocate compute by semantic risk** — that claim would contradict the mechanism our own hypothesis relies on. Any extra compute in memory-unsafe regions should instead come from the arbitrary identifiers and casts that surround pointer operations. Objective 5 tests this directly: we condition memory-specific compute profiling on identifier-token entropy via regression, and predict any apparent memory-safety effect shrinks toward zero once that confound is controlled. This is a confound-isolation test, not a search for semantic understanding; a null result here is informative, not a failed experiment.

5 Methodology

Figure 1 shows the pipeline end to end.

Model and data. We use `BlTForCausalLM` from Hugging Face `transformers` with the `itazap/blt-1b-hf` checkpoint (a 1B port of BLT’s Local Encoder $\rightarrow$ Latent Transformer $\rightarrow$ Local Decoder architecture, the largest openly available BLT checkpoint). Natural language comes from WikiText-103 and code from CodeSearchNet, filtered for Python and C++; for the prose negative control (P3) we additionally parse WikiText-103 sentences with a standard constituency parser to obtain phrase-boundary ground truth.

Boundary extraction. BLT resolves patches dynamically from the entropy threshold ($\tau \approx 1.34$) and an optional maximum patch length. The patcher exposes per-byte entropies and patch lengths directly, so boundaries are read from its output, not by instrumenting internals. Each is tagged *entropy-triggered* or *length-capped*: a patch hitting the length cap before entropy crosses τ has nothing to do with grammar, so length-capped boundaries are excluded from the primary P1/P2 scores. Entropy is reported in nats, so $\tau = 1.34 \text{ nats} = 1.93 \text{ bits}$.

Checkpoint fidelity. The only public checkpoint is a third-party conversion, so we sanity-check it before producing any numbers: coherent generation, sensible boundaries on simple inputs, and — critically — that the patcher’s effective context window and patch-length distribution match the original work. The short

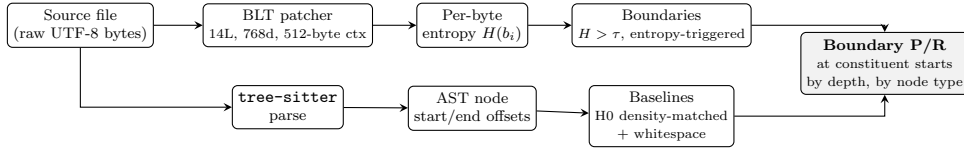


Figure 1: Extraction and scoring pipeline. The upper path is unsupervised — the patcher never sees a parse tree; the lower path supplies independently verifiable ground truth. Boundaries are scored against AST node starts and against both baselines at once, stratified by AST depth (§4.3) and node type (P4).

context window is load-bearing for §4.2; a silent mismatch would invalidate the mechanism we argue from.

AST ground truth. `tree-sitter` gives byte-level start/end offsets for statement-level nodes (`if_statement`, `for_statement`, `declaration`) and expression-level nodes (`call_expression`, `binary_expression`). Separately we query pointer dereferencing (`*`), address referencing (`&`) and manual memory management (`new/delete`) as memory-unsafe regions.

Adversarial data. A noised variant of the CodeSearchNet subset injects missing semicolons, localized indentation changes, camelCase $\leftrightarrow$ snake_case inversions, and typos in syntax keywords, to probe robustness (O4).

6 Evaluation Metrics

Boundary P/R at constituent starts (primary): precision and recall of entropy-triggered boundaries against AST node start offsets within tolerance k , reported as a margin over *both* baselines; we expect precision $>$ recall (§4.2). **P/R stratified by node type:** the same metric grouped by constituent type, separating deterministic from open-ended openers; this tests P4. **AST boundary IoU (secondary):** byte-span overlap between patches and AST nodes, expected to understate the effect per P2 and reported for comparability with prior segmentation work. **Compute density (BPP) and entropy heatmaps:** bytes-per-patch profiling and entropy visualization over source text; we expect high BPP variance in code, more uniform BPP in prose. **Degradation shift:** change in the primary metric and in BPP between clean and adversarial data, indicating robustness relative to subword tokenizers. **Memory-specific compute profiling:** BPP in memory-unsafe regions before and after conditioning on identifier-token entropy, isolating any genuine semantic-risk effect from the surface-predictability confound.

7 Feasibility, Risks and Deliverables

Feasibility and compute. The study is deliberately cheap. Everything H1 requires comes from the *patcher*: a 14-layer, 768-dimensional byte-level model of $\sim 150\text{M}$ parameters; the 1B latent transformer is never needed for boundaries. Inference in `bf16` fits a single consumer GPU and the analysis is a forward pass, so the experiment is reproducible on modest hardware; larger corpus sweeps and the adversarial condition (O4) dispatch as batch jobs on institutional compute if throughput becomes the limit. **No model training is required at any stage.**

Risks and mitigations. *Checkpoint infidelity:* context window and patch-length distribution are verified against the original configuration *before* any results are produced; this gates everything downstream. *Newline confound:* the explicit whitespace baseline (§4.3) — H1 must beat it, not merely H0. *Identifier entropy swamps syntax:* predicted in advance (§4.2) and isolated by regression in O5, so it is reported as a precision loss rather than an unmodelled confound. *Parser offset drift:* character offsets are round-tripped to UTF-8 byte offsets and validated before scoring. *Depth incomparable across languages:* alignment is computed per depth and normalized before cross-language comparison (O3).

Deliverables. (1) A boundary-extraction pipeline producing per-byte entropies, tagged boundaries and BPP statistics for arbitrary source files. (2) A `tree-sitter` AST mapping engine for Python and C++, including memory-unsafe region queries. (3) An evaluation harness implementing P1–P4, both baselines, the permutation test, and depth/node-type stratification. (4) Entropy heatmaps and BPP profiles for code and prose. (5) A final report and presentation covering all five objectives, including negative results.

8 Timeline and Division of Work

Weeks 1–3: extraction pipeline; `tree-sitter` AST mapping for Python/C++; checkpoint and context-window sanity checks — a precondition, resolved before any alignment number is generated. *Weeks 4–6:* H0 and whitespace baselines; P1/P2 alignment on code; P4 stratification; prose contrast (P3). *Weeks 7–8:* mid-submission write-up; adversarial dataset; O4. *Weeks 9–11:* memory-unsafe profiling and confound conditioning (O5); cross-language comparison (O3). *Week 12+:* consolidation, ablations, final write-up and presentation.

Division of work. Three parallel streams; analysis, writing and the presentation are shared. **Patcher and extraction:** boundary/entropy extraction, checkpoint fidelity verification, BPP profiling, entropy heatmaps. **Ground truth and data:** `tree-sitter` integration for both languages, node-type taxonomy, memory-unsafe queries, adversarial corpus construction. **Statistics and controls:** H0 and whitespace baselines, permutation testing, P3 constituency parsing, O5 confound regression.

References

- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *TACL*, 10:73–91.
- Zhangyin Feng et al. 2020. CodeBERT: A pre-trained model for

- programming and natural languages. *Findings of EMNLP 2020*, pages 1536–1547.
- John Hewitt and Christopher D. Manning. 2019. A structural probe for finding syntax in word representations. *NAAACL-HLT*, pages 4129–4138.
- Artidoro Pagnoni et al. 2024. Byte Latent Transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. *ACL*, pages 1715–1725.
- Linting Xue et al. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *TACL*, 10:291–306.